

CAPRL DICOM Listener Service

Dockerized DICOM C-STORE Receiver

1. Overview

This document describes an open-source DICOM Listener service designed to receive DICOM studies directly over the DICOM C-STORE protocol and trigger custom downstream processing workflows.

The service is packaged as a Docker container and can be deployed as a standalone DICOM ingress component in any medical imaging or healthcare AI pipeline.

It is built using:

- **DCMTK (storescp)** as the DICOM Storage SCP (Service Class Provider)
- **Python** for custom post-receipt processing logic
- **Docker** for portability and deployment

This service is intended to serve as a lightweight DICOM listener that can:

- Accept inbound DICOM C-STORE from any DICOM-compliant sender
- Persist incoming DICOM objects to local storage
- Trigger custom scripts after receiving DICOM data
- Enable integration with AI pipelines, routing workflows, metadata extraction, anonymization, and downstream DICOM forwarding

2. Architecture

2.1 Components

The service consists of the following components:

1. DCMTK **storescp**

Responsible for:

- Listening for incoming DICOM associations
- Accepting C-STORE requests
- Writing received files to disk
- Triggering post-receipt scripts

2. Python Processing Script

Responsible for:

- Reading incoming DICOM metadata
- Extracting required tags
- Executing custom business logic
- Triggering downstream workflows

3. Docker Runtime

Responsible for:

- Packaging the service
- Ensuring portability
- Standardizing deployment
- Supporting persistent volume mounts

3. Workflow

3.1 Image-Level Workflow

1. A DICOM sender opens an association with the listener
2. A DICOM C-STORE request is sent
3. `storescp` accepts the object
4. The DICOM object is written to local storage
5. The post-receipt script is triggered
6. Custom logic is executed

3.2 Study-Level Workflow

1. A DICOM sender sends all images for a study
2. The listener waits until the study is complete
3. End-of-study is detected after timeout
4. The processing script is triggered once for the completed study

4. Technology Stack

Component	Technology
DICOM Listener	DCMTK storescp
Processing Engine	Python 3.12
DICOM Parsing	pydicom
Container Runtime	Docker
Base Image	python:3.12-slim

5. Directory Structure

```
Shell
dicom-listener/
├── Dockerfile
├── entrypoint.sh
├── process_received.py
└── storage/
```

Directory Description

Path	Description
Dockerfile	Container build definition
entrypoint.sh	Container startup and DICOM listener launch
process_received.py	Custom post-receipt processing script
storage/	Persistent storage for incoming DICOM files

6. Dockerfile

None

```
FROM python:3.12-slim
```

```
ENV PYTHONUNBUFFERED=1 \  
    PYTHONDONTWRITEBYTECODE=1 \  
    DICOM_PORT=11112 \  
    DICOM_AE=CARPLSTORESCP \  
    STORAGE_DIR=/data/incoming
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \  
    dcmtool \  
    gosu \  
    ca-certificates \  
    procps \  
    && rm -rf /var/lib/apt/lists/*
```

```
RUN mkdir -p /app /data/incoming
```

```
WORKDIR /app
```

```
RUN pip install --no-cache-dir pydicom
```

```
COPY entrypoint.sh /app/entrypoint.sh  
COPY process_received.py /app/process_received.py
```

```
RUN chmod +x /app/entrypoint.sh /app/process_received.py
```

```
EXPOSE 11112
```

```
VOLUME ["/data"]
```

```
ENTRYPOINT ["/app/entrypoint.sh"]
```

7. Entrypoint Script

Shell

```
#!/bin/sh
set -e

mkdir -p "${STORAGE_DIR}"

echo "Starting DICOM Listener..."
echo "AE Title      : ${DICOM_AE}"
echo "Port          : ${DICOM_PORT}"
echo "Storage Dir     : ${STORAGE_DIR}"

exec storescp "${DICOM_PORT}" \
  --accept-all \
  --fork \
  --output-directory "${STORAGE_DIR}" \
  --filename-extension .dcm \
  --exec-sync \
  --exec-on-reception "python3 /app/process_received.py '#p/#f'
'#a' '#c'"
```

8. Processing Script

Python

```
#!/usr/bin/env python3

import sys
import json
from pathlib import Path
import pydicom

def main():
    if len(sys.argv) < 4:
        print("Usage: process_received.py <dicom_file> <calling_ae>
<called_ae>")
```

```

        sys.exit(1)

    dicom_path = Path(sys.argv[1])
    calling_ae = sys.argv[2]
    called_ae = sys.argv[3]

    try:
        ds = pydicom.dcmread(str(dicom_path),
stop_before_pixels=True)

        payload = {
            "file_path": str(dicom_path),
            "calling_ae": calling_ae,
            "called_ae": called_ae,
            "patient_id": getattr(ds, "PatientID", None),
            "patient_name": str(getattr(ds, "PatientName", "")),
            "study_instance_uid": getattr(ds, "StudyInstanceUID",
None),
            "series_instance_uid": getattr(ds, "SeriesInstanceUID",
None),
            "sop_instance_uid": getattr(ds, "SOPInstanceUID",
None),
            "modality": getattr(ds, "Modality", None),
            "accession_number": getattr(ds, "AccessionNumber",
None),
        }

        print(json.dumps(payload, indent=2))

    except Exception as e:
        print(f"Error processing {dicom_path}: {e}",
file=sys.stderr)
        sys.exit(1)

if __name__ == "__main__":
    main()

```

9. Deployment

Build

Shell

```
docker build -t dicom-listener .
```

Run

Shell

```
docker run -d \
  --name dicom-listener \
  -p 11112:11112 \
  -v $(pwd)/storage:/data \
  -e DICOM_AE=CARPLSTORESCP \
  -e DICOM_PORT=11112 \
  dicom-listener
```

10. Configuration

Variable	Description	Default
DICOM_AE	Listener AE Title	CARPLSTORESCP
DICOM_PORT	DICOM listener port	11112
STORAGE_DIR	Incoming DICOM storage path	/data/incoming

11. Study-Level Trigger Configuration (Recommended for AI)

For study-level workflows, replace:

Shell

```
--exec-on-reception "python3 /app/process_received.py '#p/#f' '#a' '#c' "
```

with:

Shell

```
--sort-conc-studies ST \  
--eostudy-timeout 15 \  
--exec-on-eostudy "python3 /app/process_received.py '#p' '#a' '#c' "
```

This ensures downstream workflows are triggered only after complete study reception.